National University of Singapore
School of Computing
CS1101S: Programming Methodology
Semester I, 2026/2027

R11
Streams

Stream Processing Functions

In addition to the Stream Processing functions of Python §3, we further define the following functions and values:

```
def add_streams(s1, s2):
    if is_none(s1):
        return s2
    elif is_none(s2):
        return s1
    else:
        return pair(head(s1) + head(s2),
                    lambda: add_streams(stream_tail(s1),
                                        stream_tail(s2)))

def mul_streams(s1, s2):
    return pair(head(s1) * head(s2),
                lambda: mul_streams(stream_tail(s1),
                                    stream_tail(s2)))

def scale_stream(c, stream):
    return stream_map(lambda x: c * x, stream)

integers = integers_from(1)
ones = pair(1, lambda: ones)
```

Problems:

1. In order to take a closer look at delayed evaluation, we will use the function `display` which displays its argument and returns it:

```
def display(x):
    print(x)
    return x
```

What is displayed in response to evaluating the following sequence?

```
x = stream_map(display, enum_stream(0, 10))
stream_ref(x, 1)
stream_ref(x, 2)
```

Explain the result using the e-stepper.

Consider the optimized version of `stream_map` covered in SICPy, using the function `memo_fun` used in Lecture L11A:

```
def memo_fun(fun):
    already_run = False
    result = None
    def mfun():
        nonlocal already_run, result
        if already_run:
            return result
        else:
            result = fun()
            already_run = True
            return result
    return mfun

def stream_map_optimized(f, s):
    return (None
            if is_none(s)
            else pair(f(head(s)),
                      memo_fun(lambda: stream_map_optimized(
                                          f, stream_tail(s)))))
```

What will be displayed when evaluating the following sequence?

```
x = stream_map_optimized(display, enum_stream(0, 10))
stream_ref(x, 1)
stream_ref(x, 2)
```

2. Consider the function `zip_streams` from Path P11A that takes as parameters two infinite streams and returns an infinite stream in which the elements of the given streams are interleaved. For example if s_1 is the infinite stream 1, 11, 111,... and s_2 is the infinite stream 2, 22, 222,... then `zip_streams(`s_1, s_2`)` returns the stream 1, 2, 11, 22, 111, 222,.... Declare the function `zip_list_of_streams` that takes a non-empty list of infinite streams

as argument and zips up them all, in the given order. For example, if s_1 and s_2 are as above and s_3 is the infinite stream 3, 33, 333,..., then `zip_list_of_streams(llist(`s_1, s_2, s_3`))` should return the stream 1, 2, 3, 11, 22, 33,....

3. Consider the function `partial_sums` from Path P11A that takes as parameter an infinite stream of numbers `s` and returns the infinite stream whose elements are s_0, s_0+s_1, $s_0+s_1+s_2$, ... For example, `partial_sums(integers)` should be the stream 1, 3, 6, 10, 15, ... Declare the function `partial_sums` using the function `add_streams` above. Does your function also work on finite streams as argument? How about the empty stream?